[Experiment, Analysis, and Benchmark] TSSMRBench: Benchmarking Memory Systems for Retrieving Correct Temporal Semantic States from Evolving Memories

Hai Lin[1,2,*], Jianhua Sun[1,*], Hai-Tao Zheng[1,2,†], Hui Wang[2]
Chunxia Ma[3,4], XiuTeng Zhou[4]
[1]Shenzhen International Graduate School, Tsinghua University, Shenzhen, China
[2]Pengcheng Laboratory, Shenzhen, China
[3]Shandong Analysis and Test Center, Qilu University of Technology
(Shandong Academy of Sciences), Jinan, China
[4]State Key Laboratory for Quality Ensurance and Sustainable Use of Dao-di Herbs,
National Resource Center for Chinese Materia Medica, China Academy of Chinese Medical Sciences, Beijing, China
{ngyygm@outlook.com, sunjianhua1225@gmail.com, zheng.haitao@sz.tsinghua.edu.cn}
wangh06@pcl.ac.cn; machunxia8927@qlu.edu.cn; zhouxt@nrc.ac.cn

***Abstract*—LLM-based agents increasingly rely on external memory systems to support long-running interactions. However, many real-world memories are not static facts: plans, preferences, project states, and event descriptions often evolve through multiple semantic versions. Existing benchmarks have advanced the evaluation of long-term memory, temporal reasoning, and knowledge update, but they rarely isolate a more specific question: can a memory system retrieve the version memory that matches a queried temporal semantic state from an evolving memory trajectory? To address this gap, we introduce TSSMRBench, a benchmark for temporal semantic state memory retrieval.**

TSSMRBench treats stage-correct version retrieval as the target capability of evaluation. We construct a benchmark with 300 scenarios, 9,000 memory nodes, and 900 multiple choice questions organized into three task families: *single-state lookup*, *cross-version comparison*, and *temporal ordering*. The tasks differ in how many supporting states they require, enabling evaluation of both retrieval precision and support completeness under shared-pool interference while holding the answer model fixed.

Experiments with BM25, FAISS, Mem0, and Graphiti show that current memory systems perform relatively well on single-state queries but degrade substantially on cross-version comparison and temporal ordering. FAISS is the strongest automatic baseline, while Mem0 and Graphiti expose different trade-offs in version fidelity, support recovery, and latency. These findings show that retrieving evolving memories remains challenging for current memory systems and suggest that future designs should preserve version history, return coherent state-level evidence, and support stage-aware retrieval over semantically similar memories.

***Index Terms*—agent memory systems, temporal semantic state retrieval, benchmark evaluation, evolving memories**

I. Introduction

Large language model agents increasingly rely on external memory systems to sustain behavior across long-running interactions. Survey work on LLM-based agent systems describes the rapid expansion of agent architectures and their growing role in complex problem solving [1]. A recent survey focused specifically on memory further argues that memory is what enables an agent to persist, organize, and selectively recall information across interactions [2]. These systems are expected to remember user preferences, track evolving plans, preserve event histories, and support multi-step decision making over time. As agents move from short exchanges to persistent assistance, memory is becoming a first-class systems component rather than a simple extension of context length.

What makes this setting difficult is that many memories evolve. A plan may be revised, a policy may be amended, and an event description may go through several intermediate stages before reaching its latest form. In such settings, the correct answer is often not the most recent memory, but the memory that was valid at a particular stage. Consider a legal assistant asked which revision of a procedural rule first introduced delayed-hearing handling, a project assistant asked to compare the current plan with an earlier draft because an old proposal is being reconsidered, or a financial agent asked to reconstruct the order of several market-state updates before issuing a decision recommendation. In each case, multiple stored memories may all be on-topic, yet only one stage or one stage set is actually compatible with the question being asked. The retrieval problem is therefore not simply whether the system reaches the right subject area, but whether it can distinguish among several closely neighboring states that remain semantically related while no longer being temporally interchangeable. These queries therefore require the memory system to recover the right *temporal semantic state*, not merely a topically relevant memory.

This requirement is not well captured by existing evaluation

* Equal contribution. † Corresponding author.

practice. Long-term memory benchmarks such as LoCoMo have substantially improved the evaluation of persistent memory [3]. Temporal and time-sensitive benchmarks such as TimeQA have sharpened our understanding of reasoning under changing information [4]. Retrieval-oriented evaluation has also advanced through RAG-style settings, for which survey work now provides a broad overview of evaluation practices and failure modes [5]. However, these benchmarks usually emphasize retention, relevance, or recency, rather than whether a memory system can distinguish among several semantically similar states of the same evolving object and retrieve the one that matches the queried stage. We discuss these neighboring benchmark families in Section 2. Here, the key point is simple: storing an evolving memory trajectory is not the same as retrieving the stage-correct state from that trajectory.

To study this problem, we introduce **TSSMRBench**, a benchmark for *temporal semantic state memory retrieval* in agent memory systems. TSSMRBench models each scenario as an evolving memory trajectory composed of temporal semantic states and evaluates whether the stage-correct state can be retrieved from a memory system given that trajectory. The benchmark contains three task families: *single-state lookup*, which targets one stage-specific state; *cross-version comparison*, which requires jointly retrieving two stage-specific states for contrast; and *temporal ordering*, which requires recovering several states and arranging them along the correct temporal progression. The central benchmark contribution is therefore not another temporal QA set, but an evaluation target centered on the memory system itself: can it recover the correct version memory for the queried temporal semantic state? Figure 1 shows one representative benchmark instance.

The current release of TSSMRBench contains 300 scenarios, 9,000 memory nodes, and 900 multiple choice questions. We evaluate representative lexical retrieval, vector-index retrieval, and memory-system baselines under a unified global mixed-pool setting, where all memories are stored together before retrieval and each question must be answered from retrieved evidence. The evaluation target is the memory system used by the agent rather than the underlying answer LLM: answer generation is held fixed so that differences primarily reflect memory retrieval quality. This setup is important because it prevents strong answer models from masking weak memory behavior and keeps the benchmark centered on whether the stored evidence returned by the memory backend is actually sufficient for the queried temporal semantic state. Our baseline design spans lexical and vector-index baselines together with recent memory-system architectures such as Mem0 [6] and Graphiti [7]. The results show that current memory systems are far more reliable on single-state questions than on comparison and ordering questions; that many failures arise from retrieving temporally incompatible but semantically adjacent states; and that more complex memory architectures do not automatically outperform simpler version-level retrievers once state fidelity, memory granularity, and latency are taken into account.

TABLE I
TSSMRBENCH AT A GLANCE. A SCENARIO IS ONE REPOSITORY-LEVEL RELEASE-EVOLUTION WINDOW, EACH MEMORY NODE IS ONE TEMPORAL SEMANTIC STATE, AND EACH SCENARIO CONTRIBUTES ONE QUESTION FOR EACH TASK FAMILY.

Item	Value
Scenarios	300 repository-level release windows
Memory nodes	9,000 temporal semantic states
Questions	900 multiple choice questions
Task families	Single-state, cross-version, temporal ordering

Our contributions are threefold:

1) We introduce TSSMRBench and formalize *temporal semantic state memory retrieval*: the ability to retrieve the stage-correct state from a memory system given an evolving memory trajectory.
2) We build a benchmark dataset with 300 scenarios, 9,000 memory nodes, and 900 questions, and organize it into three task families: single-state lookup, cross-version comparison, and temporal ordering.
3) We conduct a systematic experimental study showing that current memory systems still struggle on this problem, especially when semantically similar states coexist, and we use these findings to motivate future designs that preserve version history and support stage-aware retrieval.

Outline. Section 2 reviews related work. Section 3 presents the task definition, design principles, and benchmark construction. Section 4 reports the experimental setup and results. Section 5 concludes. Artifact availability and the required AI-use disclosure appear before the references.

II. RELATED WORK

A. Agent Memory Systems

Recent work has explored a range of designs for giving LLM agents persistent memory. MemGPT frames long-term memory as an operating-system-style resource-management problem: the LLM decides when to move information between a finite "core memory" (analogous to RAM) and a larger "archival memory" (analogous to disk), enabling persistent multi-turn interaction within a fixed context window [8]. Letta extends this line of work into a broader framework for agent memory operating across sessions and tasks [8]. Mem0 takes a more production-oriented approach: it extracts fine-grained memory facts from user interactions, stores them in a shared vector store, and retrieves the most relevant facts at query time, emphasizing scalability and ease of integration for personalized agents [6]. Graphiti, developed in the Zep ecosystem, organizes memory as a temporal knowledge graph where entities and relationships evolve over time, aiming to preserve richer structural information than flat fact stores [7]. A-MEM introduces a more agentic perspective: the memory system itself decides which memories to create, maintain, or consolidate during ongoing interaction, rather than relying on fixed extraction rules [9].

These systems differ substantially in retrieval granularity (facts, nodes, or graph episodes), memory organization (flat, hierarchical, or graph-structured), and update behavior

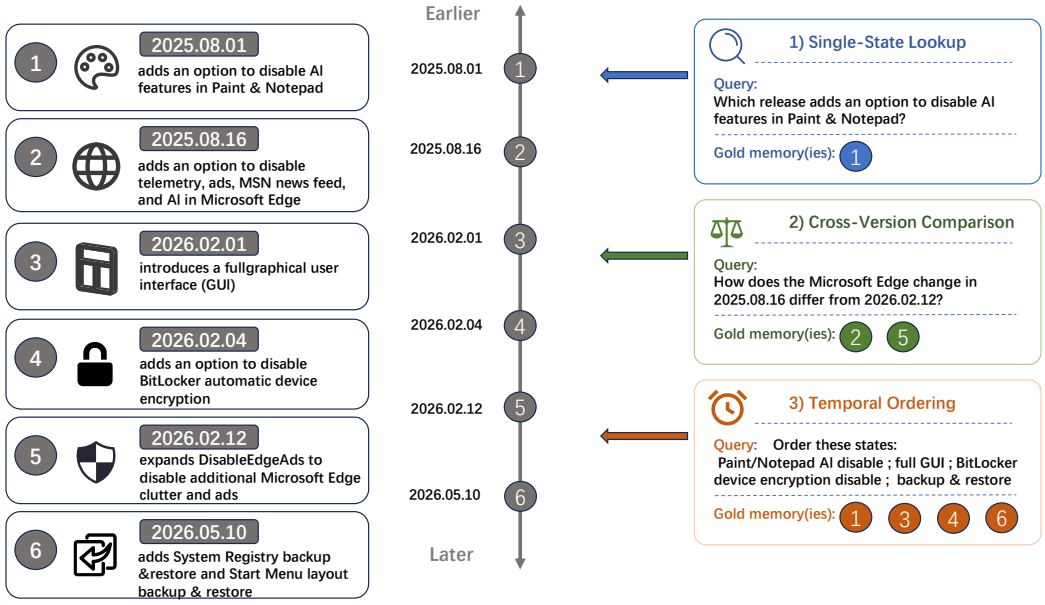

Fig. 1. Illustrative benchmark instance from the Raphire/Win11Debloat repository. The full scenario contains 30 temporal semantic states, but only the task-relevant states are shown here. The single-state lookup question depends on one state, the cross-version comparison question depends on two states, and the temporal ordering question depends on four states. The benchmark therefore tests whether a memory system can retrieve the stage-correct supporting states from an evolving memory trajectory, not merely retrieve a topically related memory.

(append-only, overwrite, or merge). However, they share a common evaluation limitation: they are typically assessed through downstream task success, user-level memory recall, or general question answering. These metrics are valuable, but they do not isolate a narrower question that is central to our benchmark: when several semantically similar states of the same evolving object coexist in memory, can the system retrieve the one that matches the queried stage, rather than the most recent or most topically relevant one? TSSMRBench is designed to test exactly this capability. By constructing explicit versioned state trajectories and asking questions that require stage-specific retrieval, our benchmark provides a targeted diagnostic that complements the broader evaluations used in these systems.

B. Long-Term Memory Benchmarks

Several recent benchmarks have advanced the evaluation of persistent agent memory. LoCoMo constructs long multi-session conversational scenarios and tests whether a system can recall facts, summaries, and event details mentioned across sessions spanning hundreds of turns [3]. LongMemEval focuses on five memory-related capabilities: knowledge retention, reasoning, forethought, understanding of temporal ordering, and handling of conflicting information, and evaluates them over distributed long histories [10]. MemoryAgentBench examines a complementary setting: agents must reuse information from prior interactions to accomplish later tasks, testing whether memory actually improves downstream performance in a task-oriented pipeline [11].

Taken together, these benchmarks have moved evaluation beyond single-turn QA and made persistent memory a serious empirical target. At the same time, they mainly treat memory as a collection of facts, dialogue details, or episodes to be recalled later. They do not usually build explicit versioned state trajectories for the same underlying object, and they rarely ask for an earlier but stage-correct state when a later state is also present. In other words, they test whether the system can *remember*, but not whether it can retrieve the *right version* of a memory that has changed over time. TSSMRBench complements this line of work by making this distinction explicit: each scenario contains a full evolution trajectory, and questions are designed so that the correct answer depends on a specific stage, not on the most recent or most similar memory. This shift matters because a memory backend that performs well on ordinary persistence may still fail once several closely related versions of the same subject coexist and the query targets only one of them.

C. Temporal Reasoning and Knowledge Update Benchmarks

A second neighboring family comes from temporal reasoning. Temporal databases and temporal query languages provided early formal tools for representing and querying time-dependent data [12], [13]. In NLP, TORQUE introduces a reading-comprehension dataset focused on event temporal ordering, testing whether models can determine which of two events happened first [14]. TimeDial studies temporal commonsense in dialogue, probing whether models understand implicit temporal relations such as duration and typical event order [15]. TimeQA targets time-sensitive question answering

over Wikipedia articles whose facts change over time, asking questions conditioned on a specific time period [4]. ETR-QA provides a comprehensive evaluation of event temporal reasoning abilities, covering multiple reasoning types such as ordering, span estimation, and hypothetical counterfactuals [16]. MusTQ introduces multi-step temporal reasoning over temporal knowledge graphs, requiring models to chain multiple temporal relationships [17]. Test of Time evaluates LLM temporal reasoning through both semantic understanding and time-arithmetic tasks, measuring whether models can perform explicit date computation [18], [19]. These benchmarks are important because they test whether models can reason about order, duration, and time-conditioned facts. However, they evaluate the *reasoning* capabilities of the model itself rather than the retrieval capabilities of the memory system that feeds the model. In TSSMRBench, the question is not whether the model can order events given their descriptions, but whether the memory system can first retrieve the right version-state evidence from a pool of semantically similar candidates.

A third neighboring family comes from retrieval and evidence-based QA. HotpotQA emphasizes multi-hop evidence aggregation, requiring systems to combine information from multiple retrieved passages [20]. NarrativeQA studies understanding over long narrative evidence, testing comprehension of plot-level events and relationships [21]. Natural Questions evaluates open-domain retrieval and answer extraction from a large web corpus using real user queries [22]. MuSiQue focuses on compositional multi-hop questions that require integrating several pieces of retrieved evidence in a specific order [23]. RAG and its later variants study how retrieved evidence can be incorporated into generation, and survey work now provides a broad overview of retrieval-augmented methods and their failure modes [5], [24]. These works sharpen our understanding of retrieval and evidence use, but they assume a static or unversioned document collection. They do not directly target stage-specific state disambiguation inside an evolving memory trajectory; that is, the retrieval target is a relevant passage, not a specific version of a changing fact.

Finally, knowledge-update settings raise a different but related issue: how a memory system should incorporate newer information and avoid outdated answers. This perspective is highly relevant, but its default assumption is often that newer information ought to dominate older information. TSSMR-Bench differs in an important way. Our goal is not simply to test whether a memory system prefers the latest state. Instead, we test whether the memory system can retrieve the *correct* state for the queried stage, even when that state is not the most recent one. This distinction matters because in many real-world scenarios (legal rule tracking, policy revision history, project plan evolution) the relevant answer may refer to an intermediate state, not the current one.

III. TSSMRBench

Figure 2 summarizes TSSMRBench. This section defines the task (Section 3.1), presents the benchmark evaluation design principles (Section 3.2), describes the data construction flow (Section 3.3) and question design (Section 3.4), and then introduces the evaluation pipeline and metrics (Section 3.5). TSSMRBench focuses specifically on whether the stage-correct state can be retrieved from a memory system given an evolving memory trajectory.

TABLE II
COMPARISON OF TSSMRBENCH WITH REPRESENTATIVE NEIGHBORING BENCHMARKS. LONGMEM INDICATES LONG-TERM MEMORY FOCUS; SPEC-STATE INDICATES EXPLICIT RETRIEVAL OF THE STAGE-CORRECT STATE; LANG. REPORTS THE PRIMARY LANGUAGE; SCALE REPORTS COARSE BENCHMARK SIZE.

Benchmark	Long Mem	Temporal	Spec-state	Lang.	Scale
LoCoMo [3]	Yes	Partial	No	EN	1,986 QA
LongMemEval [10]	Yes	Partial	No	EN	500 QA
MemoryAgentBench [11]	Yes	Partial	No	EN	Multi-source suite
TimeQA [4]	No	Yes	No	EN	≈40K QA
MusTQ [17]	No	Yes	No	EN	≈666K QA
TSSMRBench	Yes	Yes	Yes	EN	300 repositories / 900 QA

A. Task Definition

TSSMRBench evaluates *temporal semantic state memory retrieval*: whether the stage-correct state can be retrieved from a memory system given an evolving memory trajectory, rather than merely returning a relevant or recent memory.

Formally, let

$$\mathcal{T} = (m_1, m_2, \ldots, m_n)$$

denote an evolving memory trajectory, where each memory node m_i records a semantic state of the same underlying object, event, entity, or plan, and where the ordering function

$$\tau(m_i) < \tau(m_j)$$

indicates that m_i precedes m_j in the evolution process. A natural-language query q specifies, either explicitly or implicitly, a stage constraint $\sigma(q)$ over this trajectory. Let

$$G(q) \subseteq \mathcal{T}$$

be the minimal set of supporting state nodes sufficient to answer q under $\sigma(q)$, and let

$$R_k(q) = (r_1, r_2, \ldots, r_k)$$

be the ranked retrieval output returned by a memory system.

The task is to retrieve a ranked list $R_k(q)$ such that the supporting set $G(q)$ is recovered and the answer generator can derive the gold answer $y(q)$ using only the retrieved evidence. In plain terms, the memory system receives a query about a specific stage of an evolving object and must return exactly those memory nodes that describe the object at that stage. The core difficulty is that several states in $\mathcal{T}$ may be semantically similar while being temporally incompatible with the query. For example, if a project has gone through ten planning revisions, the memory system must distinguish between the version that first introduced a feature, the version

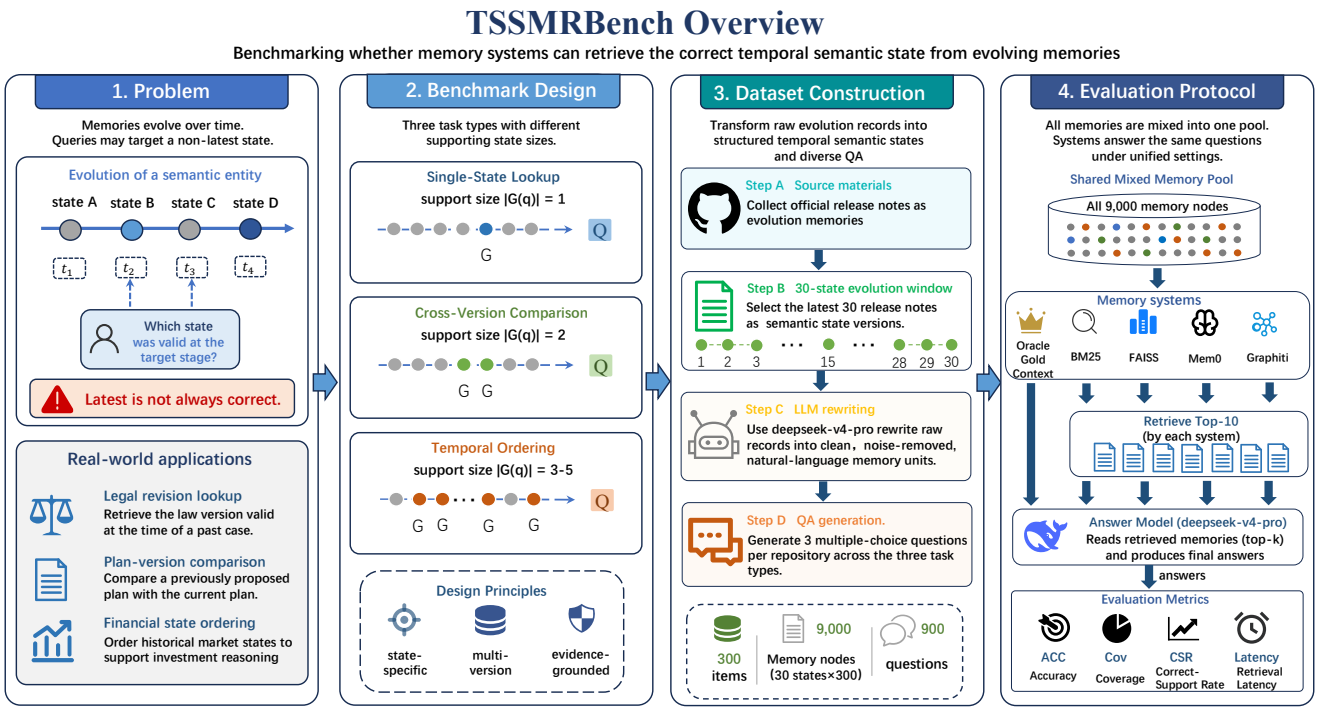

Fig. 2. Overall framework of TSSMRBench. The figure summarizes the temporal semantic state retrieval problem, the benchmark task design, the release-derived data construction pipeline, and the unified evaluation protocol. It is intended as a paper-level overview of how the benchmark connects the problem definition, dataset construction, and memory-system evaluation.

that later modified it, and the current version, even though all ten revisions may describe the same feature in similar language. The formal task definition therefore treats temporal semantic state retrieval as a problem of selecting the correct version-conditioned support set, not merely returning evidence that remains topically close to the query surface.

TSSMRBench operationalizes this capability through three task families:

- **Single-state lookup**: $|G(q)| = 1$. The memory system must identify the one state node that supports the answer.
- **Cross-version comparison**: $|G(q)| = 2$. The memory system must recover the two states required to contrast what changed.
- **Temporal ordering**: $|G(q)| \in \{3, 4, 5\}$. The memory system must recover enough states to reconstruct a local evolution order.

These tasks differ in support size, but they test the same underlying ability: retrieving the stage-correct state or state set required by the query.

B. Benchmark Evaluation Design Principles

To isolate this capability, the benchmark and its evaluation protocol follow five principles.

a) Stage-specific validity.: Questions must depend on which temporal semantic state is valid for the queried stage, not simply on topical relevance or recency. For instance, a question asking "which version first introduced delayed-hearing handling?" requires the system to identify the earliest matching state, not the latest one.

b) Evidence-grounded answering.: The final answer should only be derivable by aligning the question's state description with retrieved memory content, rather than by exploiting superficial cues such as version labels, dates, or option formatting. This prevents the answer model from guessing the correct option without actually reading the retrieved evidence.

c) Varied support requirements.: Some questions depend on one state while others require several states. By varying the support size across task families, the benchmark can distinguish pointwise retrieval (whether the system can find one correct state) from complete support recovery over a trajectory, which requires finding all of the relevant states simultaneously.

d) Mixed-memory interference.: All retrieval operates over repository-mixed memory collections rather than isolated per-trajectory stores. This means the memory system cannot exploit trajectory boundaries to narrow the search space, and must instead disambiguate the correct state from a much larger pool of semantically overlapping memories.

e) Separating retrieval failure from state-selection failure.: The protocol distinguishes whether a memory system missed the required state entirely (retrieval failure) or instead

retrieved semantically related but temporally incompatible evidence (state-selection failure). This distinction is important because the two failure modes suggest very different system improvements: retrieval failures call for better indexing or recall strategies, while state-selection failures call for better temporal reasoning or version-aware reranking.

C. Data Construction Flow

We instantiate these principles using an auditable release-only benchmark built from official repository evolution records.

a) Step 1: Data source selection.: We use official GitHub release notes because they are repository-authoritative descriptions of evolving software states, including feature additions, policy changes, fixes, migrations, and deprecations. Compared with informal discussions such as commit messages or issue threads, release notes provide a cleaner and more reproducible record of how a system changes across versions. They are also publicly available and version-stamped, which makes the benchmark fully auditable: any result can be traced back to a specific release of a specific repository.

b) Step 2: Trajectory node determination.: For each repository, we take the latest 30 usable release notes and treat them as a fixed local evolution window. Each release note becomes one temporal semantic state node, producing an evolving memory trajectory of length 30. We choose 30 nodes because it is long enough to create meaningful interference among semantically similar states (the memory system must pick out the correct stage from dozens of candidates) while still being short enough for the evaluation pipeline to process reliably. Repositories without a sufficient release history are excluded. The formal benchmark contains 300 repositories, yielding 9,000 memory nodes in total.

c) Step 3: LLM rewriting of raw release notes.: Raw release notes contain hyperlinks, formatting markers, references, and other artifacts that are not ideal as a memory surface. We therefore use a large language model to rewrite each release note into a normalized English memory unit. This rewriting preserves repository identity and release-specific state content while removing markup-heavy noise. The normalization serves two purposes: it puts all memory units on a level playing field so that retrieval systems are not biased by differences in formatting style across repositories, and it removes information that would make the task trivially easy (e.g., direct links to earlier releases). The resulting normalized text is the only memory surface ingested by the evaluated memory systems; the original release note is retained only for audit and provenance.

d) Step 4: QA generation.: On top of each 30-node evolving memory trajectory, we generate one multiple choice question for each task family, yielding 3 questions per repository and 900 questions overall. For every question, we annotate the supporting state nodes required for answering it. This annotation step is essential: without it, we could only measure final answer correctness, but with it we can also measure whether the memory system retrieved the right supporting states regardless of whether the final answer was correct. This makes it possible to evaluate not only final answer correctness but also support recovery.

D. Question Design

Question design is driven by support complexity rather than by superficial wording diversity.

a) Single-state lookup.: These questions ask the memory system to identify one state node that contains a target condition, decision, fix, or release-specific behavior. For example, a question might ask "which release first introduced an option to disable AI features in Paint & Notepad?" The memory system must locate the one state node among 30 candidates that describes this specific behavior, rather than returning any state that merely mentions AI features. They test whether the memory system can localize a single correct state among semantically nearby alternatives.

b) Cross-version comparison.: These questions require contrasting two states that differ in a clear, evidence-supported way. For example, a question might ask "what changed between release X and release Y regarding the handling of BitLocker encryption?" Answering requires retrieving both states and identifying the concrete difference between them. We only construct such questions when the source evidence contains a substantive difference between the two releases. Comparison questions are not generated from pairs whose behavior is essentially unchanged.

c) Temporal ordering.: These questions require mapping several state descriptions back to their supporting nodes and ordering them correctly along the trajectory. For example, a question might present four feature descriptions and ask which temporal ordering is correct. The difficulty is that the descriptions themselves do not contain explicit dates or version numbers; instead, the answer model must retrieve the corresponding memory nodes and use their content to determine which came first. They are designed so that solving them robustly still depends on retrieving the corresponding memory content, rather than simply relying on superficial ordering cues in the question text alone.

All questions are multiple choice. Version identifiers, tags, or publication cues may appear when they are part of the original material, but the question and options are constrained so that the answer cannot be derived reliably without aligning the state descriptions in the query to the stored memory units.

E. Benchmark Record Schema

Beyond task design, TSSMRBench is released as an auditable benchmark rather than only as a set of final scores. Each scenario contains both the evolving memory trajectory and the associated questions, and the released records preserve the link between source evidence, normalized memory text, annotated support, and evaluation output. This design matters for a benchmark of evolving memories because reviewers and future users need to inspect not only whether a system was right or wrong, but also which version states were available,

TABLE III
TASK FAMILIES AND EVALUATION BUDGETS IN TSSMRBENCH. $|G(q)|$ DENOTES THE NUMBER OF SUPPORTING STATES REQUIRED BY A QUESTION, AND RETRIEVAL TOP-K LISTS THE ANSWER-TIME CUTOFFS REUSED FROM ONE SHARED TOP-10 RETRIEVAL RESULT.

Task family	$\|G(q)\|$	Retrieval top-k
Single-state lookup	1	1, 2, 3
Cross-version comparison	2	2, 5, 8
Temporal ordering	3–5	5, 8, 10

TABLE IV
RELEASED RECORD TYPES IN TSSMRBENCH AND THEIR ROLES IN AUDITABILITY.

Record type	Key fields	Role in the benchmark
State node	repository, release or tag, timestamps, raw release note, normalized memory unit	Preserves both source provenance and the normalized retrieval surface for each temporal semantic state.
Question record	task type, options, correct option, supporting node ids, answer evidence	Specifies the query target and the gold supporting states needed for stage-correct answer evaluation.
Evaluation record	retrieved memory text, matched node ids, support coverage, per-k answers, retrieval latency	Makes system behavior traceable from final score back to retrieved evidence and support recovery.

which ones were annotated as support, and what the system actually retrieved.

At the memory level, each released state node preserves repository identity, release identity, timestamps, the original release note, and the normalized memory unit used as the retrieval surface. Retaining both the raw and normalized forms allows the benchmark to remain faithful to the original source while still standardizing the memory surface presented to each baseline. At the question level, every record stores the task type, options, correct option, and the annotated supporting node identifiers. These supporting identifiers are essential because they turn the benchmark from pure answer evaluation into memory-system evaluation: they allow us to measure not only final answer correctness, but also whether the system retrieved the stage-correct version states required by the query.

This schema also supports the failure analysis reported later in Section 4. Because the released evaluation records retain retrieved memory text, matched state identifiers, support coverage, and per-k answer outputs, we can trace any reported result back to a concrete retrieval event. This is especially important for TSSMRBench, where many failures arise not from complete topic mismatch but from selecting the wrong version within a semantically similar set of states. A benchmark that claims to diagnose version-sensitive retrieval should therefore expose enough intermediate evidence for such distinctions to be inspected directly.

F. Evaluation Pipeline and Metrics

All baselines are evaluated under a shared global memory pool: before any query is issued, all 9,000 memory nodes from all 300 repositories are written into each baseline's own memory backend or retrieval index. Retrieval therefore operates over a single repository-mixed memory collection per baseline, rather than over isolated per-trajectory memory stores. This design choice is intentional: in a real deployed agent, memory from all interactions would coexist in the same store, and the retrieval system would need to disambiguate among overlapping memories without knowing which repository or context they belong to. This setting introduces cross-repository interference and prevents the memory system from exploiting trajectory boundaries. As a result, the evaluation reflects not only whether a system can recover the right state within one local version history, but also whether it can preserve that state's identity under realistic mixed-pool competition from many unrelated yet partially overlapping memories.

For each query q, the evaluated memory system retrieves the top-10 ranked memories:

$$R_{10}(q) = (r_1, \ldots, r_{10}).$$

The same ranked list is then reused across task-specific context budgets:

- **single-state lookup**: $k \in \{1, 2, 3\}$
- **cross-version comparison**: $k \in \{2, 5, 8\}$
- **temporal ordering**: $k \in \{5, 8, 10\}$

The answer model is allowed to read only these retrieved memories and is not given access to unretrieved memory nodes. This protocol separates retrieval quality from how much of the ranked memory list is exposed at answer time. It also allows us to analyze whether a memory system places the required support near the top of the ranking or only recovers it deeper in the retrieved list. In other words, a system is rewarded not only for eventually reaching the right repository or topic, but for ranking the stage-correct version evidence high enough to remain usable under realistic answer-time context limits.

We report four primary metrics. **Accuracy (ACC)** measures whether the final answer is correct; this is the end-to-end metric that reflects what a user would observe. **Coverage (Cov)** measures how much of the required support set is recovered. In other words, of all the gold supporting states needed to answer the question, what fraction did the memory system actually retrieve:

$$\mathrm{Cov}(q, k) = \frac{|G(q) \cap R_k(q)|}{|G(q)|}.$$

Complete Support Rate (CSR) measures whether the full support set is recovered: a binary indicator that is 1 only when every required supporting state appears in the retrieved list.

$$\mathrm{CSR}(q, k) = \mathbf{1}[G(q) \subseteq R_k(q)].$$

Latency measures retrieval time only, excluding answer generation.

Because TSSMRBench is intended to diagnose memory retrieval rather than only final QA success, these metrics are designed to jointly capture answer correctness, support completeness, and retrieval cost.

TABLE V
SYSTEMS AND CONFIGURATIONS USED IN THE FORMAL BENCHMARK. ORACLE GOLD CONTEXT BYPASSES RETRIEVAL AND DIRECTLY SUPPLIES THE ANNOTATED SUPPORTING MEMORY UNITS TO THE ANSWER MODEL.

System	Retrieval unit	Internal memory model	Embedding / reranking	Answer model
BM25	memory unit text	none	lexical matching	DeepSeek-V4-Pro
FAISS	memory unit text	none	BGE-M3 embeddings	DeepSeek-V4-Pro
Mem0	memory facts	DeepSeek-V4-Flash	BGE-M3 embeddings	DeepSeek-V4-Pro
Graphiti	graph episodes / nodes	DeepSeek-V4-Flash	BGE-M3 embeddings + BGE Reranker v2 M3	DeepSeek-V4-Pro
Oracle Gold Context	annotated supporting memory units	none	none	DeepSeek-V4-Pro

IV. EXPERIMENTS

A. Experimental Setup

The formal benchmark is built from 300 repositories, yielding 300 repository-level release-evolution scenarios, 9,000 memory nodes, and 900 multiple choice questions. We compare five baselines under the same benchmark inputs: BM25, FAISS, Mem0, Graphiti, and Oracle Gold Context.

BM25 indexes the normalized memory units directly and serves as a lexical release-level retriever. FAISS is a vector-index baseline over the same release-level memory units. Mem0 is an LLM-based memory system that extracts fine-grained memory facts from each ingested state, stores them in a vector-backed memory store, and applies memory merging and update operations; at query time, it returns grouped fact results, which we align back to the corresponding source state nodes for node-level metrics. Graphiti is the graph-structured memory processing package used in the Zep line of work: it uses an internal LLM to extract episodes, entities, and relations, updates a structured memory graph over time, and reranks retrieved candidates before returning them. Oracle Gold Context does not perform retrieval; instead, it directly provides the answer model with the annotated supporting memory units required by each question. We include Oracle Gold Context as an ideal-retrieval control: it serves as an upper bound on end-to-end answer accuracy when the required supporting states are supplied directly.

All baselines use the same normalized memory units and the same answer-generation model. In the final evaluation, answer generation and LLM-based judging both use DeepSeek-V4-Pro with temperature 0.0 and thinking disabled. The embedding-based baselines use BGE-M3 embeddings. Mem0 and Graphiti use DeepSeek-V4-Flash as their internal memory-processing model, and Graphiti additionally uses a BGE Reranker v2 M3 model. The answer model is therefore controlled across baselines, so the evaluation target is the memory system used by the agent rather than the answer LLM itself.

B. Overall Results

Table VII reports the main results. Oracle Gold Context is treated as ideal retrieval, so its Cov and CSR are 1.0 by construction because the annotated supporting states are provided directly to the answer model. Its remaining errors appear only in ACC. The Oracle scores therefore provide a control upper bound on answer accuracy under ideal retrieval and make it possible to quantify how much room remains once the correct supporting states are guaranteed to be available.

TABLE VI
TASK-WISE ANSWER ACCURACY AT THE PRIMARY RETRIEVAL TOP-K OF EACH TASK FAMILY.

System	Single (k=3)	Cross (k=8)	Temporal (k=10)
BM25	0.8800	0.3933	0.3200
FAISS	0.9433	0.8400	0.6300
Mem0	0.6567	0.4467	0.2600
Graphiti	0.7267	0.6700	0.4767
Oracle Gold Context	1.0000	0.9800	0.9100

Among the automatic baselines, FAISS is the strongest overall reference point. It achieves the best non-oracle ACC, Cov, and CSR on most task and retrieval-top-k settings, and it remains the most stable as the benchmark moves from single-state lookup to cross-version comparison and temporal ordering. This pattern is important for interpreting the benchmark: a comparatively simple vector-index baseline over complete version-level memory units is already strong, so improvements from dedicated memory systems must come from preserving version fidelity better than this baseline, not merely from adding more internal processing.

The remaining systems reveal where current memory architectures still struggle. BM25 is competitive on easier single-state questions but degrades quickly on multi-state tasks, where lexical overlap is too weak a proxy for version identity. Mem0 trails FAISS and Graphiti across all three task families, with the largest drops on cross-version comparison and temporal ordering. This matches its underlying mechanism: the system stores extracted facts rather than intact version states, and its memory merging and update behavior can further blur or overwrite version-history signals that the benchmark later requires. Graphiti is stronger than BM25 and Mem0 on the harder tasks and even exceeds FAISS on cross-version comparison at k=2 (ACC 0.5467 vs. 0.5033), showing that graph organization and reranking can help when the retrieval budget is tight. However, its overall results still remain below FAISS, which indicates that richer structure alone does not guarantee preservation of the exact version boundary needed by these questions.

C. Results by Task Type and Top-k

Figure 3 reports the results by task family and retrieval top-k. The first trend concerns larger retrieval top-k. Coverage usu-

TABLE VII

MAIN RESULTS ON TSSMRBENCH. EACH CELL REPORTS ACC / COV / CSR, WHERE ACC IS ANSWER ACCURACY, COV IS THE AVERAGE FRACTION OF GOLD SUPPORTING STATES RECOVERED IN THE RETRIEVED SET, AND CSR IS THE FRACTION OF QUESTIONS FOR WHICH THE FULL GOLD SUPPORT SET IS RECOVERED. EACH TASK FAMILY IS EVALUATED AT ITS OWN RETRIEVAL TOP-k SETTINGS: SINGLE-STATE LOOKUP USES k=1, 2, 3; CROSS-VERSION COMPARISON USES k=2, 5, 8; TEMPORAL ORDERING USES k=5, 8, 10. ORACLE GOLD CONTEXT IS SHOWN AS AN UPPER BOUND AND IS EXCLUDED FROM STRONGEST-BASELINE MARKING. BOLD MARKS THE BEST AUTOMATIC BASELINE AND UNDERLINE MARKS THE SECOND-BEST AUTOMATIC BASELINE FOR EACH METRIC UNDER THE SAME TASK AND TOP-k SETTING.

		Retrieval top-k		
Task	System	k=1	k=2	k=3
Single-state lookup	BM25	0.8533 / 0.8333 / 0.8333	0.8633 / 0.8733 / 0.8733	0.8800 / 0.8900 / 0.8900
	FAISS	**0.8933 / 0.8667 / 0.8667**	**0.9133 / 0.9200 / 0.9200**	**0.9433 / 0.9500 / 0.9500**
	Mem0	0.6433 / 0.6733 / 0.6733	0.6533 / 0.7033 / 0.7033	0.6567 / 0.7100 / 0.7100
	Graphiti	0.6800 / 0.6467 / 0.6467	0.7300 / 0.7167 / 0.7167	0.7267 / 0.7267 / 0.7267
	Oracle Gold Context	0.9967 / 1.0000 / 1.0000	1.0000 / 1.0000 / 1.0000	1.0000 / 1.0000 / 1.0000
		k=2	k=5	k=8
Cross-version comparison	BM25	0.2100 / 0.2767 / 0.1233	0.3200 / 0.3933 / 0.2567	0.3933 / 0.4483 / 0.3033
	FAISS	0.5033 / 0.6033 / 0.3433	**0.7467 / 0.8067 / 0.6467**	**0.8400 / 0.8917 / 0.7933**
	Mem0	0.2700 / 0.4400 / 0.1900	0.4067 / 0.5917 / 0.3833	0.4467 / 0.6400 / 0.4567
	Graphiti	**0.5467 / 0.6200 / 0.3933**	0.6567 / 0.7150 / 0.5100	0.6700 / 0.7417 / 0.5467
	Oracle Gold Context	0.9900 / 1.0000 / 1.0000	0.9833 / 1.0000 / 1.0000	0.9800 / 1.0000 / 1.0000
		k=5	k=8	k=10
Temporal ordering	BM25	0.2767 / 0.5406 / 0.2033	0.3267 / 0.5803 / 0.2600	0.3200 / 0.6094 / 0.2867
	FAISS	**0.4633 / 0.7036 / 0.3033**	**0.5633 / 0.8200 / 0.5200**	**0.6300 / 0.8606 / 0.6233**
	Mem0	0.1867 / 0.5556 / 0.1733	0.2500 / 0.6136 / 0.2733	0.2600 / 0.6356 / 0.3067
	Graphiti	0.3767 / 0.6503 / 0.2733	0.4833 / 0.6975 / 0.3533	0.4767 / 0.7136 / 0.3767
	Oracle Gold Context	0.9067 / 1.0000 / 1.0000	0.9100 / 1.0000 / 1.0000	0.9100 / 1.0000 / 1.0000

ally increases with k, especially on cross-version comparison and temporal ordering, but answer accuracy does not rise at the same rate. This pattern means that recovering more gold states is not the same as making the final decision easier. In many questions, larger top-k adds neighboring versions that are topically related but not all stage-correct; in Mem0, it also adds more fact fragments that point to the same repository topic without cleanly reconstructing the required version states. The answer model therefore sees more relevant evidence, but also more competing evidence, so ACC can plateau or even decline while Cov keeps rising.

The second trend is a consistent difficulty ordering across baselines: single-state lookup is easiest, cross-version comparison is harder, and temporal ordering is hardest. This ordering matches the benchmark design. As the number of required supporting states grows from one to two and then to three-to-five, the retrieval target is no longer "a relevant memory" but "the temporally compatible subset of version states needed by the question." The relative behavior of the baselines follows the same logic. FAISS remains strongest because each retrieved item is still a complete version-level memory unit, so the answer model sees intact states rather than fragments. BM25 declines more sharply because lexical overlap is too coarse to separate nearby versions with similar wording. Mem0 improves less because its fact-level representation and update pipeline do not naturally preserve complete version boundaries. Graphiti benefits from structure and reranking, especially on cross-version comparison, but its gains flatten once the question requires several exact states rather than one relevant neighborhood.

TABLE VIII

MIXED-EFFECTS LOGISTIC REGRESSION ON ANSWER CORRECTNESS AT THE MAIN TASK-SPECIFIC RETRIEVAL TOP-K SETTINGS. THE MODEL USES FIXED EFFECTS FOR TASK FAMILY AND SYSTEM, WITH A RANDOM INTERCEPT FOR QUESTION. ODDS RATIOS ARE REPORTED RELATIVE TO SINGLE-STATE LOOKUP.

Contrast	Odds ratio	95% interval
Cross vs Single	0.26	[0.23, 0.30]
Temporal vs Single	0.11	[0.09, 0.12]

To quantify the task effect beyond the descriptive plots, we fit a mixed-effects logistic regression on answer correctness using the automatic baselines at the main task-specific retrieval top-k settings. The model uses fixed effects for task family and system, with a random intercept for question. Table VIII shows that, relative to single-state lookup, the odds of a correct answer drop to $0.26\times$ for cross-version comparison and to $0.11\times$ for temporal ordering. This model-based result supports the same conclusion suggested by the curves: current memory systems are much less reliable once the query depends on multiple temporally related states rather than one isolated state.

D. Decoupling Retrieval from Reasoning

To better diagnose failure modes, we decouple retrieval success from final answer correctness. For each question, we categorize the retrieval outcome into three buckets: *all-recalled*, where all annotated supporting states are retrieved; *partial-recalled*, where only part of the support set is retrieved;

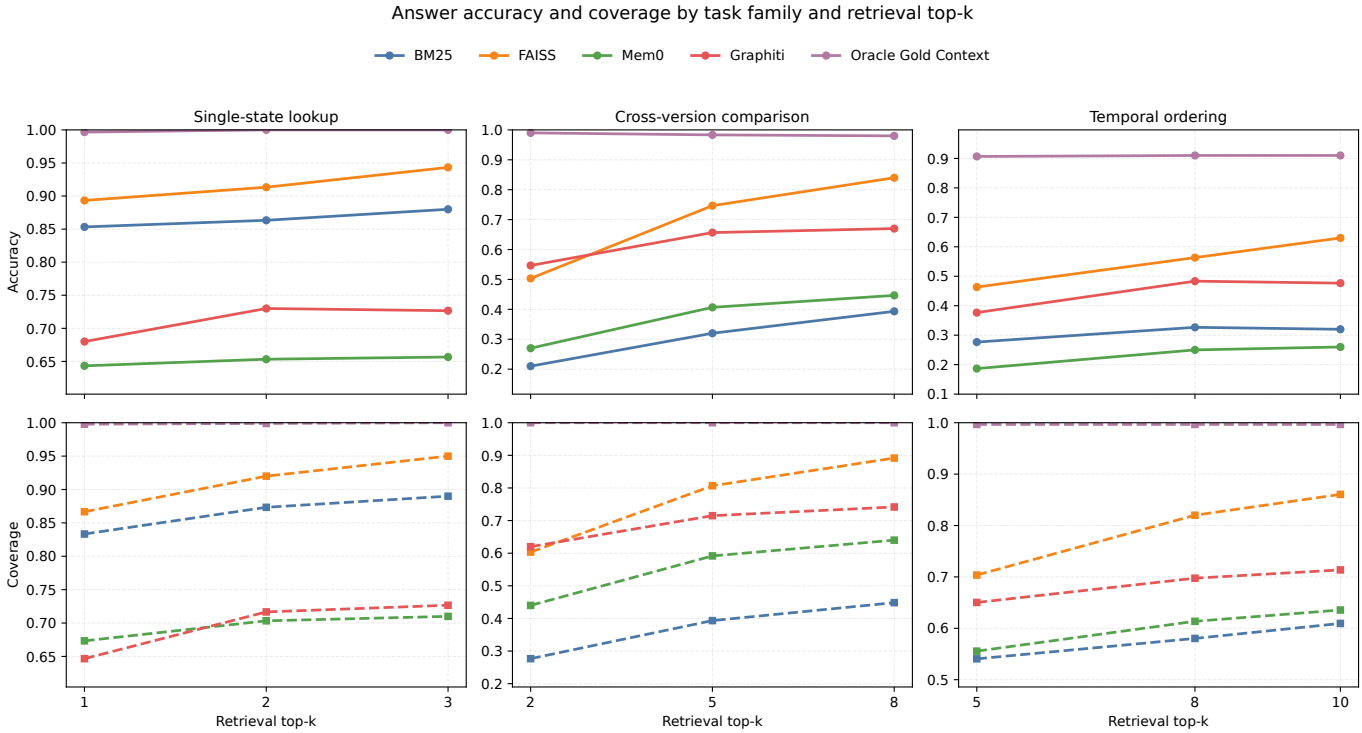

Fig. 3. Answer accuracy and gold-support coverage by task family and retrieval top-k. The top row reports answer accuracy and the bottom row reports gold-support coverage; the three columns correspond to single-state lookup, cross-version comparison, and temporal ordering.

and *zero-recalled*, where none of the supporting states are retrieved. We then split each bucket by whether the final answer is correct or incorrect.

This analysis is important because overall answer accuracy alone cannot tell us whether a memory system fails by missing relevant memories entirely, by retrieving incomplete support, or by retrieving related but temporally incompatible evidence. In particular, the *all-recalled but incorrect* bucket is highly diagnostic: it indicates that the memory system has already retrieved the necessary state evidence but the final answer is still not resolved correctly. Conversely, the *partial-recalled but correct* and *zero-recalled but correct* buckets indicate cases where the answer may be guessed from partial cues or option bias rather than supported by complete evidence.

Figure 4 shows that the baselines fail for different reasons, and the intermediate retrieval outputs make these differences concrete. FAISS has the strongest retrieval profile: 73.44% of all questions fall into the all-recalled and correct bucket, while only 2.00% fall into zero-recalled and incorrect. Its typical failure is not missing the topic entirely, but returning several nearby versions and leaving the final temporal discrimination unresolved. For example, in the FuelLabs/sway temporal-ordering question on the progression of four feature states, the retrieved context already contains the required versions such as v0.67.0, v0.68.0, v0.66.7, and v0.71.1, yet the final answer is still wrong. This is exactly the boundary that TSSMRBench is designed to expose: retrieving the correct

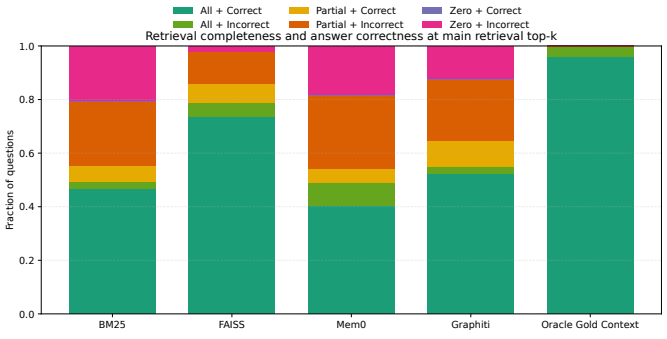

Fig. 4. Retrieval–reasoning decoupling at the main task-specific retrieval top-k settings. Each bar partitions questions by retrieval completeness and final answer correctness.

version set is necessary, but the benchmark is hard because several semantically adjacent versions may all look relevant at once.

Mem0 presents a different profile once its grouped fact results are aligned back to source state nodes. Only 0.22% of all questions fall into zero-recalled and correct, confirming that Mem0 rarely succeeds without relevant evidence. Its dominant error buckets are partial-recalled and incorrect (27.44%) and all-recalled but incorrect (8.89%). The retrieval logs show why. In the 2dust/v2rayNG comparison question about TLS changes in releases 2.0.8 and 2.2.2, Mem0 retrieves 2.2.2 but misses

TABLE IX
REPRESENTATIVE FAILURE CASES FROM THE OFFICIAL RETRIEVAL LOGS AT THE MAIN TASK-SPECIFIC RETRIEVAL TOP-k SETTINGS.

System	Query	Retrieved gold	Diagnosis
Mem0	v2rayNG cross-version	1/2 states (2.2.2 found; 2.0.8 missed)	Retrieval drifts toward neighboring TLS-related releases such as 2.0.18, 2.1.8, 2.0.13, and 2.0.1. The returned fact groups are topic-relevant but do not recover the full comparison pair.
Graphiti	HelloGitHub cross-version	1/2 states (vol.109 found; vol.110 missed)	Retrieval reaches the correct repository neighborhood, but the candidate set is dominated by nearby issues such as vol.101, vol.102, vol.103, vol.105, and vol.106 rather than the exact contrasting state.
FAISS	sway ordering	4/4 states recovered	Retrieval succeeds, but the returned set also contains six adjacent versions. The failure is therefore not recall loss; it is an evidence-integration error after the correct version set has already been retrieved.

TABLE X
PER-TASK RETRIEVAL COMPLETENESS AT THE MAIN TASK-SPECIFIC RETRIEVAL TOP-K SETTINGS. EACH CELL REPORTS THE FRACTION OF QUESTIONS IN THE ALL-RECALLED, PARTIAL-RECALLED, AND ZERO-RECALLED BUCKETS.

Task	System	All	Partial	Zero
Single	BM25	0.89	0.00	0.11
	FAISS	0.95	0.00	0.05
	Mem0	0.71	0.00	0.29
	Graphiti	0.73	0.00	0.27
Cross	BM25	0.30	0.29	0.41
	FAISS	0.79	0.20	0.01
	Mem0	0.46	0.37	0.18
	Graphiti	0.55	0.39	0.06
Temporal	BM25	0.29	0.61	0.11
	FAISS	0.62	0.37	0.00
	Mem0	0.31	0.61	0.09
	Graphiti	0.38	0.59	0.04

2.0.8, while returning nearby releases 2.0.18, 2.1.8, 2.0.13, and 2.0.1. This is a retrieval miss caused by similarity among related TLS facts, not a reasoning failure over a complete evidence set. In other failures, Mem0 reaches node-level full coverage after alignment but still returns fact bundles that foreground other details from the same release, which leaves the answer model without a clean version-level contrast. The overall pattern is consistent with its mechanism: fact extraction remains useful for topical recall, but fact storage plus memory merging/update can weaken the coherence of the original version state.

Graphiti suggests a third failure mode. It is stronger than Mem0 on cross-version comparison and temporal ordering, which shows the value of structured retrieval, but it still has a large partial-recalled and incorrect segment (22.78%). In the 521xueweihan/HelloGitHub comparison between vol.110 and vol.109, Graphiti retrieves vol.109 together with nearby issues such as vol.101, vol.102, vol.103, vol.105, and vol.106, but omits the missing gold state vol.110. The retrieved region is therefore repository-correct and topic-relevant, yet still incomplete for the required version comparison. This behavior is consistent with its mechanism: graph expansion and reranking can steer retrieval toward the right neighborhood, but they do not guarantee that the exact stage-correct node survives candidate selection.

Table X makes the same point from the task perspective. On single-state lookup, the partial-recalled bucket is empty by construction because the gold support contains exactly one state. On cross-version comparison and temporal ordering, partial-recalled becomes the dominant bucket for most systems. This is important because it shows that many failures on TSSMRBench are not caused by retrieving nothing at all. Instead, the system often reaches the right topic or repository, but still fails to recover the full version set required by the query. BM25 reveals a particularly instructive pattern: its zero-recalled rate is low on single-state lookup (0.11), rises sharply on cross-version comparison (0.41), and then drops again on temporal ordering (0.11). This low–high–low pattern is consistent with lexical matching. When only one support state is needed, an exact lexical cue is often enough for BM25 to hit the target. When two exact versions must be recovered jointly, lexical overlap becomes brittle: nearby releases often share most of the same wording, so BM25 can drift to non-gold neighbors and miss both required versions entirely. On temporal ordering, however, the gold set contains three to five states, so BM25 often recovers at least one lexically similar state even when it cannot assemble the whole temporally correct set; the result is fewer zero-recalled cases but many more partial-recalled ones. FAISS retains the highest all-recalled rate across all three tasks, yet it still drops from 0.95 on single-state lookup to 0.62 on temporal ordering. Mem0 and Graphiti exhibit a different contrast: Graphiti more often reaches a relevant retrieval region on temporal ordering, but Mem0 and Graphiti remain similarly constrained by incomplete multi-state recovery once several stage-correct versions must be assembled together.

Taken together, these findings sharpen the paper's main point. The challenge is not only to store evolving memories, but to preserve version identity strongly enough that the memory system can later retrieve the stage-correct state. TSSMRBench therefore evaluates a capability that lies between general topical retrieval and full temporal reasoning: retrieving the correct version memory for a queried temporal semantic state.

E. Computational Efficiency

The previous sections focused on retrieval quality. However, for any memory system that must operate in real time, latency and cost are equally important. This section examines the computational efficiency of each baseline.

Table XI shows the results. FAISS offers the strongest efficiency–effectiveness trade-off among the automatic baselines: it is the fastest retriever (195.12 ms on average) and also the strongest automatic system in accuracy, coverage, and complete-support rate. BM25 is slower than FAISS despite weaker overall effectiveness, so it remains mainly a lexical reference point.

Mem0 occupies a different point in the trade-off space. Its latency remains moderate, and its retrieved contexts are the shortest among the automatic systems because it operates on

compact memory facts. However, the main results show that this compactness comes with lower state-level fidelity: the system is efficient at retrieving small fact snippets, but those snippets often do not reconstruct a complete version state. Graphiti sits at the opposite end. It retrieves contexts of similar scale to FAISS on the harder tasks, but its retrieval latency is much higher (34,425.76 ms on average), which matches its heavier pipeline of graph expansion, LLM-mediated memory processing, and reranking.

For this benchmark, the efficiency conclusion is straightforward. A memory system that aggressively compacts memory can become fast but lose the version structure required by the question. A memory system that preserves richer structure can recover harder cases more often, but the added processing must remain affordable at retrieval time. Future memory systems therefore need both properties together: explicit preservation of version history and efficient retrieval of the stage-correct state from a large shared memory pool.

Taken together, the retrieval traces and the system-level results suggest three concrete design requirements for future memory systems. First, memory compaction should remain traceable to coherent version-level states. The strongest baseline in this study is not the one with the richest internal processing, but the one that preserves each temporal semantic state as a complete retrieval unit. Second, memory update should preserve version history instead of folding successive states into a blurred summary. Mem0's failures show that fact extraction plus merging can retain topical clues while weakening the exact state boundary needed by cross-version and temporal-ordering questions. Third, retrieval should support explicit multi-state assembly rather than only nearest-neighbor matching. Many failures on TSSMRBench occur after the system has already reached the right repository or topic, but before it has assembled the full temporally compatible support set required by the question.

These requirements also clarify the benchmark's broader value. TSSMRBench is not simply harder because it contains long histories or mixed repositories; it is diagnostic because it isolates a failure mode that is easy to miss in ordinary memory QA. A system may appear to remember the right subject, return highly related evidence, and even retrieve the latest state correctly, yet still fail when the query requires a particular earlier state or a coordinated set of states along the same version history. The benchmark therefore exposes a gap between remembering that something evolved and retrieving which state of that evolution is correct for the current question.

TABLE XI
COMPUTATIONAL COST PROFILE. LATENCY IS MEAN RETRIEVAL-ONLY LATENCY; TOKEN COLUMNS REPORT THE MEAN ANSWER-CONTEXT TOKENS AT THE PRIMARY RETRIEVAL TOP-k OF EACH TASK FAMILY.

System	Latency	Single	Cross	Temporal
BM25	1110.63	181.41	402.16	438.14
FAISS	195.12	270.17	611.98	856.29
Mem0	417.71	90.72	174.46	214.38
Graphiti	34425.76	224.58	539.11	836.39
Oracle Gold Context	0.00	87.64	169.43	309.78

V. CONCLUSION

We presented TSSMRBench, a benchmark for temporal semantic state memory retrieval. Its target is precise: given an evolving memory trajectory, can a memory system retrieve the version memory that matches the queried temporal semantic state, rather than merely a relevant topic or the most recent state.

To study this ability, we built a benchmark with 300 release-evolution scenarios, 9,000 temporal semantic states, and 900 questions spanning single-state lookup, cross-version comparison, and temporal ordering. This design turns stage-correct version retrieval into a directly testable benchmark problem and separates it from ordinary topical retrieval.

Our experiments show that current memory systems remain far stronger on single-state lookup than on questions that require multiple temporally related states. FAISS is the strongest automatic baseline because it retrieves complete version-level memory units effectively, but it still degrades when several nearby versions are all relevant. Mem0 is limited by fact-level extraction together with memory merging and update, which often weakens version-state fidelity. Graphiti benefits from structured retrieval on some harder questions, yet it still frequently retrieves the right neighborhood without recovering the full stage-correct version set. These results show that retrieving evolving memories is not enough: the central challenge is preserving and recovering the correct version state.

The main design implication is clear. Future memory systems should not only store evolving information, but also preserve version history, return coherent state-level evidence, distinguish among semantically similar stages, and do so efficiently in large shared memory pools. TSSMRBench provides a benchmark for measuring that capability directly.

ARTIFACT AVAILABILITY

The source code, benchmark dataset, evaluation outputs, and paper materials are publicly available at https://github.com/ngyygm/llm-TSSMRBench.

ACKNOWLEDGMENTS

The authors used ChatGPT for limited English-language editing to improve the fluency, grammar, and clarity of text originally drafted by the authors. ChatGPT was also used during code development to assist with routine implementation tasks. In addition, as part of the benchmark methodology itself, large language models were used in the construction and evaluation pipeline, as described in the main text: DeepSeek-V4-Flash was used for internal memory processing in Mem0 and Graphiti, while DeepSeek-V4-Pro was used for memory-unit rewriting, question construction, and answer generation under retrieved evidence. Answer evaluation was performed by scripts. No research findings, analyses, or scientific claims were generated solely by AI tools. The authors take full responsibility for the accuracy, originality, and integrity of all content presented in this paper.

REFERENCES

[1] T. Guo, X. Chen, Y. Wang, R. Chang, S. Pei, N. V. Chawla, O. Wiest, and X. Zhang, "Large language model based multi-agents: A survey of progress and challenges," in *Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence*, 2024, pp. 8048–8057. [Online]. Available: https://doi.org/10.24963/ijcai.2024/890

[2] P. Du, "Memory for autonomous LLM agents: Mechanisms, evaluation, and emerging frontiers," 2026. [Online]. Available: https://arxiv.org/abs/2603.07670

[3] A. Maharana, D.-H. Lee, S. Tulyakov, M. Bansal, F. Barbieri, and Y. Fang, "Evaluating very long-term conversational memory of LLM agents," in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Bangkok, Thailand: Association for Computational Linguistics, 2024, pp. 13 851–13 870. [Online]. Available: https://doi.org/10.18653/v1/2024.acl-long.747

[4] W. Chen, X. Wang, and W. Y. Wang, "A dataset for answering time-sensitive questions," 2021, accepted to NeurIPS 2021 Datasets and Benchmarks Track. [Online]. Available: https://arxiv.org/abs/2108.06314

[5] Y. Gao, Y. Xiong, X. Gao, K. Jia, J. Pan, Y. Bi, Y. Dai, J. Sun, M. Wang, and H. Wang, "Retrieval-augmented generation for large language models: A survey," 2023. [Online]. Available: https://arxiv.org/abs/2312.10997

[6] P. Chhikara, D. Khant, S. Aryan, T. Singh, and D. Yadav, "Mem0: Building production-ready AI agents with scalable long-term memory," 2025. [Online]. Available: https://arxiv.org/abs/2504.19413

[7] P. Rasmussen, P. Paliychuk, T. Beauvais, J. Ryan, and D. Chalef, "Zep: A temporal knowledge graph architecture for agent memory," 2025. [Online]. Available: https://arxiv.org/abs/2501.13956

[8] C. Packer, S. Wooders, K. Lin, V. Fang, S. G. Patil, I. Stoica, and J. E. Gonzalez, "MemGPT: Towards LLMs as operating systems," 2023. [Online]. Available: https://arxiv.org/abs/2310.08560

[9] W. Xu, Z. Liang, K. Mei, H. Gao, J. Tan, and Y. Zhang, "A-Mem: Agentic memory for LLM agents," in *Advances in Neural Information Processing Systems*, vol. 38, 2025. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2025/hash/19909c36f51abc4856b4560aff3d36d6-Abstract-Conference.html

[10] D. Wu, H. Wang, W. Yu, Y. Zhang, K.-W. Chang, and D. Yu, "LongMemEval: Benchmarking chat assistants on long-term interactive memory," in *The Thirteenth International Conference on Learning Representations*, 2025. [Online]. Available: https://openreview.net/forum?id=pZiyCaVuti

[11] Y. Hu, Y. Wang, and J. McAuley, "Evaluating memory in LLM agents via incremental multi-turn interactions," in *The Fourteenth International Conference on Learning Representations*, 2026. [Online]. Available: https://openreview.net/forum?id=DT7JyQC3MR

[12] C. S. Jensen and R. T. Snodgrass, "Temporal data management," *IEEE Transactions on Knowledge and Data Engineering*, vol. 11, no. 1, pp. 36–44, 1999. [Online]. Available: https://doi.org/10.1109/69.755613

[13] TSQL2 Language Design Committee, *The TSQL2 Temporal Query Language*. Boston, MA: Kluwer Academic Publishers, 1995. [Online]. Available: https://www2.cs.arizona.edu/~rts/tsql2.html

[14] Q. Ning, H. Wu, R. Han, N. Peng, M. Gardner, and D. Roth, "TORQUE: A reading comprehension dataset of temporal ordering questions," in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing*. Online: Association for Computational Linguistics, 2020, pp. 1158–1172. [Online]. Available: https://doi.org/10.18653/v1/2020.emnlp-main.88

[15] L. Qin, A. Gupta, S. Upadhyay, L. He, Y. Choi, and M. Faruqui, "TIMEDIAL: Temporal commonsense reasoning in dialog," in *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*. Online: Association for Computational Linguistics, 2021, pp. 7066–7076. [Online]. Available: https://doi.org/10.18653/v1/2021.acl-long.549

[16] S. Luo, Y. Liu, D. Lin, Y. Zhai, B. Wang, X. Yang, and J. Liu, "ETRQA: A comprehensive benchmark for evaluating event temporal reasoning abilities of large language models," in *Findings of the Association for Computational Linguistics: ACL 2025*. Vienna, Austria: Association for Computational Linguistics, 2025, pp. 23 321–23 339. [Online]. Available: https://doi.org/10.18653/v1/2025.findings-acl.1198

[17] T. Zhang, J. Wang, Z. Li, J. Qu, A. Liu, Z. Chen, and H. Zhi, "MusTQ: A temporal knowledge graph question answering dataset for multi-step temporal reasoning," in *Findings of the Association for Computational Linguistics: ACL 2024*. Bangkok, Thailand: Association for Computational Linguistics, 2024, pp. 11 688–11 699. [Online]. Available: https://doi.org/10.18653/v1/2024.findings-acl.696

[18] B. Fatemi, S. M. Kazemi, A. Tsitsulin, K. Malkan, J. Yim, J. Palowitch, S. Seo, J. Halcrow, and B. Perozzi, "Test of time: A benchmark for evaluating LLMs on temporal reasoning," in *The Thirteenth International Conference on Learning Representations*, 2025. [Online]. Available: https://openreview.net/forum?id=44CoQe6VCq

[19] S. Wei, W. Li, F. Song, W. Luo, T. Zhuang, H. Tan, Z. Guo, and H. Wang, "TIME: A multi-level benchmark for temporal reasoning of LLMs in real-world scenarios," 2025. [Online]. Available: https://arxiv.org/abs/2505.12891

[20] Z. Yang, P. Qi, S. Zhang, Y. Bengio, W. Cohen, R. Salakhutdinov, and C. D. Manning, "HotpotQA: A dataset for diverse, explainable multi-hop question answering," in *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*. Brussels, Belgium: Association for Computational Linguistics, 2018, pp. 2369–2380. [Online]. Available: https://doi.org/10.18653/v1/D18-1259

[21] T. Kočiský, J. Schwarz, P. Blunsom, C. Dyer, K. M. Hermann, and G. Melis, "The NarrativeQA reading comprehension challenge," *Transactions of the Association for Computational Linguistics*, vol. 6, pp. 317–328, 2018. [Online]. Available: https://doi.org/10.1162/tacl_a_00023

[22] T. Kwiatkowski, J. Palomaki, O. Redfield, M. Collins, A. Parikh, C. Alberti, D. Epstein, I. Polosukhin, J. Devlin, K. Lee, K. Toutanova, L. Jones, M. Kelcey, M.-W. Chang, A. Dai, J. Uszkoreit, Q. Le, and S. Petrov, "Natural questions: A benchmark for question answering research," *Transactions of the Association for Computational Linguistics*, vol. 7, pp. 452–466, 2019. [Online]. Available: https://doi.org/10.1162/tacl_a_00276

[23] H. Trivedi, N. Balasubramanian, T. Khot, and A. Sabharwal, "MuSiQue: Multihop questions via single-hop question composition," *Transactions of the Association for Computational Linguistics*, vol. 10, pp. 539–554, 2022. [Online]. Available: https://doi.org/10.1162/tacl_a_00475

[24] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems*, vol. 33. Virtual: Curran Associates, Inc., 2020, pp. 9459–9474. [Online]. Available: https://proceedings.neurips.cc/paper/2020/hash/6b493230205f780e1bc26945df7481e5-Abstract.html